

한국어 챗봇 응답 생성(Q→A)

결과보고서

토큰화·임베딩 설계부터 전이학습 파인튜닝, 디코딩 전략·하이퍼파라미터 비교,
정량 평가와 오류 분석까지의 전 과정과 의사결정 근거

항목	내용
과제	한국어 단문 대화 데이터로 질문(Q)에 대한 응답(A)을 생성하는 모델 설계·학습·평가
데이터	dataset/ChatbotData.xlsx — 11,823쌍 (Q, A, 감정 label 3종)
베이스라인	직접 구현한 인코더-디코더 Transformer (처음부터 학습, 6.0M 파라미터)
전이학습	KoBART(gogamza/kobart-base-v2, 123.9M) / KE-T5(KETI-AIR/ke-t5-base, 247.5M) 파인튜닝
최종 결과	전이학습(KoBART) BLEU 7.90 vs 베이스라인 4.87 — 전이학습이 62% 우세
산출물	노트북 8종(01~08), 결과 지표 CSV/JSON, 본 보고서
작성일	2026-09-19
실행 환경	Windows 11 · Python 3.11 · PyTorch 2.11.0+cu128 · NVIDIA RTX 4070 Ti

0. 요약

이 프로젝트는 "전이학습이 실제로 무엇을 얼마나 개선하는가"를 같은 데이터·같은 지표로 증명하는 것을 하나의 줄거리로 삼아 설계했다. 이를 위해 처음부터 학습하는 Transformer를 베이스라인으로 두고, 한국어로 사전학습된 KoBART/KE-T5를 파인튜닝한 모델을 비교군으로 두었다. 두 계열 모두 같은 전처리·같은 test set·같은 평가 기준을 적용했다.

총 8개의 노트북으로 진행했으며, 각 단계는 앞 단계에서 확보한 근거 숫자 위에서 다음 선택을 하도록 이어져 있다. 예를 들어 01번에서 측정한 길이 95 percentile이 03번의 MAX_LEN 근거가 되고, 02번에서 비교한 토큰화 단위 결론이 07번 평가 지표의 토큰화 기준으로 다시 쓰인다.

최종 결과

모델	계열	BLEU	ROUGE-L	Distinct-2	학습 epoch
KoBART (lr=5e-5 + warmup)	전이학습	7.90	25.14	0.510	7
baseline-114ep (직접 구현)	베이스라인	4.87	22.99	0.303	117
KE-T5 (lr=5e-5 + warmup)	전이학습	0.09	19.47	0.003	7 (수렴 실패)

표 0-1. 최종 성능 요약 (test 1,175건, Okt 형태소 기준 채점).

전이학습이 베이스라인을 BLEU 기준 62% 앞섰고(7.90 vs 4.87), 다양성(Distinct-2)도 68% 높았다. 학습 비용은 오히려 7 epoch 대 117 epoch으로 훨씬 적었다. 정확도와 다양성을 동시에 끌어올렸다는 점이 중요하다 — 보통 이 둘은 상충한다.

주요 과정과 발견

- 데이터 11,823쌍 중 (Q, A) 완전 중복 73건만 제거해 11,750쌍 확보, label 비율을 유지한 채 8:1:1로 분할(9,400 / 1,175 / 1,175).
- 토큰화는 어절·음절·형태소·자모 네 단위를 실측 비교한 뒤, OOV 대응을 이유로 서브워드(SentencePiece BPE, vocab 8,000)를 베이스라인 토큰나이저로 채택했다. 전이학습 모델에는 각 모델 고유 토큰나이저를 그대로 사용했다.
- 베이스라인(30 epoch)은 같은 토큰을 반복하는 생성 붕괴를 보였고, 다른 조건을 고정한 채 epoch 상한만 늘린 통제 실험(117 epoch 조기 종료)으로 그 원인이 학습 부족이었음을 확인했다 — 단일 토큰 반복 8.3% → 1.4%, 응답 다양성 4.7배.
- 가장 큰 성능 개선은 모델 변경이 아니라 결함 수정에서 나왔다. 아래 참조.

이 프로젝트의 핵심 발견

정량 지표가 정성적 인상과 어긋났을 때, 지표를 의심하는 대신 파이프라인을 의심했다.

07번 최초 평가에서 사전학습 KoBART(BLEU 3.83)가 베이스라인(4.87)에 지는 예상 밖 결과가 나왔다. "챗봇은 정답이 여러 개니까 지표의 한계겠지"로 넘어갈 수도 있었지만, 원인을 끝까지 추적한 결과 파인튜닝 타깃에 EOS(문장 끝) 토큰이 빠져 있어 모델이 문장을 끝내는 법 자체를 학습하지 못한 결함을 발견했다(KoBART 응답의 정상 종결률 32.3%).

EOS 하나를 고치고 재학습하자 BLEU가 3.83 → 7.72로 2배 이상 올랐고, 결론이 뒤집혔다. 베이스라인과 KE-T5라는 두 대조군의 수치가 전혀 변하지 않아 인과도 명확하다.

부수적으로, 디코딩 단계의 **no_repeat_ngram** 제약이 반복을 없애준다는 최초의 결론도 재해석됐다 — 그것은 학습 결함을 덮는 임시방편이었고, 원인을 고치자 사실상 무효가 됐다. 증상을 덮는 대신 원인을 찾는 것이 옳은 해법이었다.

1. 과제 정의와 전체 설계

1.1 무엇을 증명해야 하는 과제인가

과제 요구사항을 한 문장으로 요약하면 "토큰화·임베딩·전이학습을 어떻게, 왜 선택했는지 근거를 대고, 베이스라인 대비 무엇이 얼마나 개선됐는지 실험과 오류분석으로 증명하라"이다. 즉 결과 수치 하나를 내는 것이 목적이 아니라, 선택의 근거와 개선의 인과를 보이는 것이 목적이다.

그래서 프로젝트 전체를 처음부터 하나의 대비 구조로 설계했다.

베이스라인 = 사전학습 없이 처음부터 학습하는 Transformer
고도화(전이학습) = 사전학습된 KoBART/KE-T5를 파인튜닝

이 둘을 동일한 데이터, 동일한 평가 지표로 비교해 "전이학습이 실제로 무엇을 얼마나 개선하는가"를 수치와 오류 사례로 보여주는 것이 이 프로젝트의 핵심 줄거리다.

이 줄거리를 따라가면 각 노트북이 왜 그 순서로 존재하는지가 자연스럽게 설명된다. 또한 각 단계는 "다른 조건은 고정하고 하나만 바꾼다"는 통제 실험 원칙을 지켜, 개선이 관찰될 때 그 원인을 특정할 수 있도록 했다.

1.2 전체 파이프라인 흐름

단계	노트북	핵심 질문	다음 단계로 넘기는 것
1	01_data_edu	데이터가 어떻게 생겼고 무엇을 정제해야 하는가	정제·분할된 train/val/test
2	02_tokenization_embedding	어떤 분석 단위로 쪼개고 어떤 임베딩을 쓸 것인가	BPE 토큰라이저, Word2Vec
3	03_baseline_transformer	어떤 구조를 왜 선택하고, 처음부터 학습하면 어디까지 되는가	베이스라인 체크포인트, 관찰된 문제
4	04_finetune_pretrained	반복 붕괴의 원인이 학습 부족인가 / 전이학습은 얼마나 좋은가	재학습 베이스라인, KoBART
5	05_decoding_strategy	반복 문제는 디코딩 전략으로 얼마나 해결되는가	대표 디코딩 설정
6	06_hparam_comparison	하이퍼파라미터/모델을 바꾸면 무엇이 달라지는가	하이퍼파라미터 변형 모델 6종
7	07_evaluation	위의 모든 선택이 정량 지표로도 맞는 선택이었나	모델별 지표, 전체 생성 결과
8	08_error_analysis	그 수치가 나온 이유는 무엇이고 무엇이 남았는가	오류 유형 분포, 대표 실패 사례

표 1-1. 노트북 단계별 역할. 각 단계는 앞 단계의 산출물을 입력으로 받아 이어진다.

1.3 각 단계에서 내린 핵심 의사결정 한눈에

의사결정	선택	근거 요약
중복 제거 기준	(Q, A) 쌍이 완전히 같을 때만 제거	Q만 같고 A가 다른 것은 "같은 질문에 여러 답"이라는 자연스러운 다양성
max_length 기준	길이 분포의 95 percentile	최댓값 기준은 이상치 때문에 대부분의 짧은 문장에 padding 낭비
임베딩 학습 단위	형태소(Ok)	의미 단위 보존과 시퀀스 길이 사이의 절충점 (실측 비교 결과)
모델 입력 토큰화	서브워드 BPE (vocab 8,000)	형태소 분석기는 신조어·오타에 취약, BPE는 OOV가 사실상 없음

의사결정	선택	근거 요약
사전학습 토큰나이저	모델 고유 AutoTokenizer 그대로	사전학습 임베딩이 그 vocab index에 종속 — 바꾸면 사전학습 지식이 무의미해짐
모델 구조	인코더-디코더	Q→A는 조건부 생성. 소스/타겟 경계가 구조적으로 분리되어 소규모 데이터에 유리
모델 크기	d_model 256 / layer 3 (원 논문의 절반)	데이터 9,400쌍에 원 논문 크기는 파라미터 과잉
평가 지표	BLEU·ROUGE-L·Distinct-n·빈 응답률	챗봇은 정답이 여러 개 — 표면 유사도 하나로는 오판 위험
평가 토큰화	형태소 단위	어절 기준은 조사 차이에 과민, 음절 기준은 과도하게 관대

표 1-2. 프로젝트 전반의 주요 의사결정과 근거. 각 항목의 상세 근거는 해당 장에서 다룬다.

2. [01] 데이터 탐색 및 전처리

2.1 왜 EDA를 가장 먼저 하는가

EDA를 첫 단계로 둔 이유는 단순한 관례가 아니다. 뒤 단계에서 반드시 정해야 하는 값들 — 토큰라이저의 vocab size, 모델의 max_length, 평가 시 label별 성능 해석 기준 — 이 모두 여기서 측정한 숫자를 근거로 삼기 때문이다. 예를 들어 Q/A 길이의 95 percentile을 몰랐다면 03번의 MAX_LEN을 감으로 정했을 것이다.

2.2 데이터 개요와 품질 검사

항목	값
원본 규모	11,823행 × 3열 (Q, A, label)
결측치	Q/A/label 모두 0건
전체 컬럼 기준 중복	0건
(Q, A) 기준 중복	73건
Q만 기준 중복 (답변은 다름)	161건
label 분포	0 일상 5,290 / 1 이별(부정) 3,570 / 2 사랑 2,963
Q 길이	평균 12.88음절 · 3.59어절, 95p 25음절, 최대 56음절
A 길이	평균 15.02음절 · 3.69어절, 95p 27음절, 최대 76음절

표 2-1. 원본 데이터 기초 통계 (01번 노트북 실행 결과).

중복 제거 기준을 (Q, A) 쌍으로 정한 이유

중복은 세 가지 기준으로 각각 세어봤다. 전체 컬럼 기준으로는 0건, (Q, A) 기준으로는 73건, Q만 기준으로는 161건이었다. 이 중 (Q, A) 기준 73건만 제거하기로 했다. 판단 근거는 다음과 같다.

- Q만 같고 A가 다른 경우(88건)를 남긴 이유: "같은 질문에 여러 답이 있을 수 있다"는 것은 대화 데이터의 자연스러운 성질이고, 오히려 응답 다양성을 학습시키는 데 도움이 된다.
- (Q, A)가 완전히 같은 경우를 제거한 이유: 추가 정보 없이 특정 패턴의 손실 가중치만 키워 그 패턴으로 편향시킨다.

다만 제거 대상을 실제로 눈으로 확인하는 과정에서 경계 사례를 발견했다. (Q, A)는 완전히 같은데 label만 다른 행들이 있었다 — 예를 들어 "결혼이나 하지 왜 자꾸 나한테 화 내냐구!" → "힘들겠네요."가 label 0과 label 1로 각각 존재한다. 현재는 label을 학습에 쓰지 않고 평가 시 분해 기준으로만 사용하므로 제거해도 무방하다고 판단했으나, 향후 label을 조건으로 주는 모델로 확장한다면 이 행들은 "같은 Q-A가 두 감정 맥락에서 모두 성립한다"는 정보이므로 재검토가 필요하다. 이 사항은 미해결 항목으로 남겨둔다.

2.3 길이 분포와 max_length 원칙

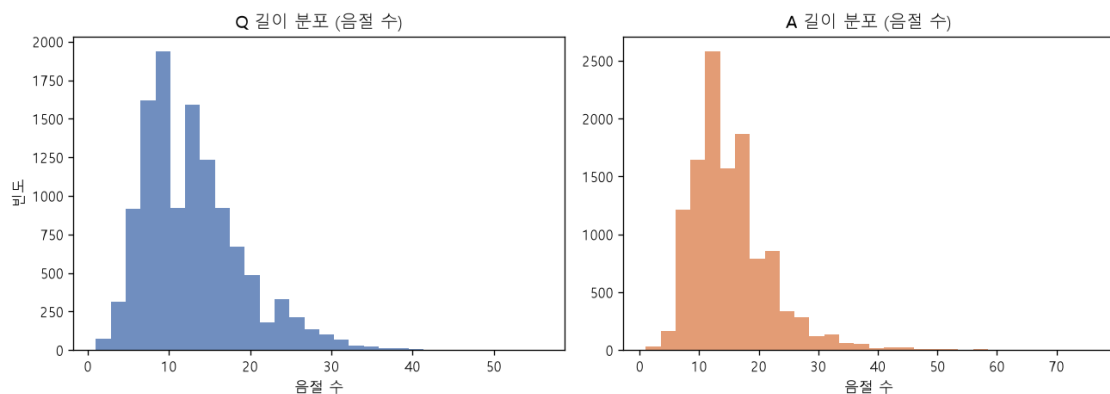


그림 2-1. Q/A 길이 분포 (음절 기준). 짧은 쪽에 몰려 있고 긴 꼬리가 존재한다.

왜 최댓값이 아니라 95 percentile을 max_length 기준으로 쓰는가: 히스토그램에서 보이듯 길이 분포는 짧은 쪽에 몰려 있고 극소수의 긴 문장이 긴 꼬리를 만든다. max_length를 최댓값(Q 56, A 76음절)으로 잡으면 그 이상치 때문에 나머지 대부분의 짧은 문장까지 전부 그 길이로 padding되어 연산과 메모리가 낭비된다. 반대로 평균으로 잡으면 상당수 문장이 잘려 정보가 손실된다. 95 percentile은 "상위 5%의 일부 잘림을 감수하고 나머지 95%는 손실 없이 담는" 절충점이다.

이 원칙은 03번에서 실제 MAX_LEN을 정할 때 같은 방식으로 다시 적용했다. 다만 그때는 음절이 아니라 실제 모델 입력 단위인 BPE 토큰 길이로 다시 측정했고(Q/A 모두 95p = 10토큰), 여기에 BOS/EOS 2토큰을 더한 뒤 GPU 연산 효율을 위해 8의 배수로 올려 MAX_LEN = 16을 얻었다.

2.4 분할 전략

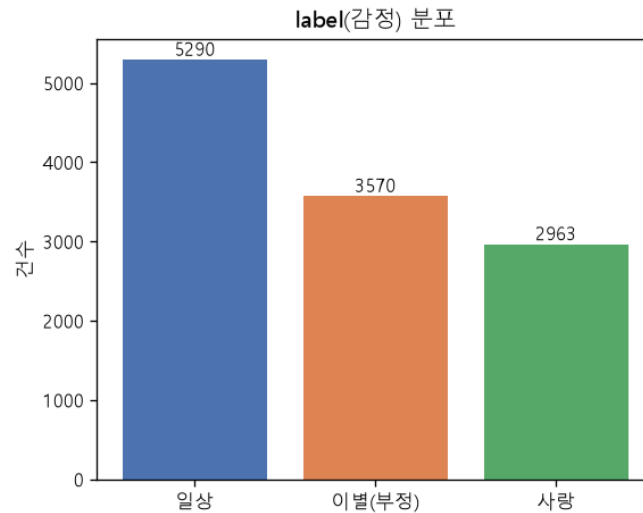


그림 2-2. label(감정) 분포. 일상(0)에 쏠려 있다.

정제 후 11,750건을 8:1:1로 나눠 train 9,400 / val 1,175 / test 1,175건을 얻었다. 분할 시 label 비율을 유지(stratify)한 이유는 위 그림처럼 label 분포가 균등하지 않기 때문이다. 무작위로 나누면 test set의 label 구성이 train과 달라져, 07번에서 label별 성능을 분해할 때 "모델이 약한 것"과 "그 label 샘플이 적은 것"을 구분할 수 없게 된다.

3. [02] 토큰화 및 임베딩

3.1 네 가지 분석 단위를 모두 비교한 이유

"형태소 단위를 쓴다"는 결론이 감이 아니라 실측 위에 서 있어야 하므로, 한국어에서 쓸 수 있는 네 가지 분석 단위를 같은 문장에 모두 적용해 평균 토큰 수를 잰다(train Q 500문장 표본).

분석 단위	평균 토큰 수	특징	문제점
어절 (공백 분리)	3.53	구현이 가장 단순	조사·어미가 붙어 있어 vocab이 커지고 희소해짐
형태소 (Okt)	5.26	의미 단위를 보존	규칙·사전 기반이라 신조어/오타에 취약
음절 (글자)	10.31	vocab이 매우 작음(~2,350자)	음절 하나는 대개 의미를 갖지 않아 표현력 부족
자모	24.54	오타자·신조어에 가장 강인	시퀀스가 매우 길어져 self-attention 연산량 급증

표 3-1. 분석 단위별 평균 토큰 수 실측 결과 (02번 노트북).

토큰 수가 많을수록 Transformer의 연산 비용(시퀀스 길이의 제곱에 비례)과 학습 난이도가 커지므로, 무조건 잘게 쪼갬다고 좋은 것이 아니다. 형태소가 의미 보존과 시퀀스 길이 사이의 절충점이라는 결론을 이 실측으로 확인했고, 이후 Word2Vec 임베딩 학습 단위와 07번 평가 지표의 토큰화 기준으로 재사용했다.

3.2 그런데 왜 모델 입력은 형태소가 아니라 서브워드(BPE)인가

형태소가 절충점이라는 결론과 모델 입력 토큰나이저 선택은 별개의 문제다. Okt 같은 형태소 분석기는 사전·규칙 기반이라 학습 데이터에 없던 신조어·오타·외래어에서 분석이 깨진다. 이를 실제 예시로 확인했다.

입력 문장	Okt 형태소 분석	BPE 서브워드 분해
오늘 급여루팡했다	['오늘', '급여', '루팡', '했다']	[' ', '오늘', ' ', '급', ' ', '여', ' ', '루', ' ', '팡', ' ', '했다']
르ㅇㅋㅋ 완전 갓생살기	['르ㅇㅋㅋ', '완전', '갓', '생', '살기']	[' ', '르', 'ㅇ', 'ㅋ', 'ㅋ', ' ', '완전', ' ', ' ', '갓', ' ', '생', ' ', '살', ' ', '기']

표 3-2. OOV(미등록어) 대응 비교. 형태소 분석기는 신조어를 의미 없는 단위로 쪼개거나 잘못 분석한다.

BPE는 문자 병합 통계만으로 vocab을 만들기 때문에 이론상 모든 문자열을 서브워드 조합으로 표현할 수 있어 OOV 문제가 사실상 사라진다. 이는 다음 단계에서 쓸 사전학습 Transformer(BERT/GPT/BART/T5, KoBART 포함)가 예외 없이 서브워드를 쓰는 이유와 정확히 같다.

SentencePiece를 선택한 이유 (WordPiece 대비)

WordPiece는 공백으로 단어를 먼저 나눈 뒤 그 안에서 서브워드를 만들지만, SentencePiece는 원문을 공백 포함 raw text로 그대로 다룬다. 띄어쓰기가 불규칙한 구어체 한국어("안녕하세요" / "안녕 하세요")에 안정적이라는 뜻이다. 또한 파인튜닝 대상인 KoBART/KE-T5가 모두 SentencePiece 계열이라, 베이스라인도 같은 방식으로 맞추면 이후 비교에서 "토큰나이저 방식 차이" 변수를 제거할 수 있다.

vocab_size = 8,000의 근거

학습 코퍼스는 train의 Q와 A를 각각 한 줄로 쓴 18,800줄이다. vocab을 3만 이상으로 크게 잡으면 저빈도 서브워드가 많아져 학습이 비효율적이고 임베딩 파라미터만 늘어난다. 반대로 1천 이하로 작게 잡으면 지나치게 잘게 쪼개져 시퀀스가 길어진다. 소규모 대화체 코퍼스에서 흔히 쓰이는 범위(8,000~16,000)의 하단인 8,000을 선택했다.

3.3 가장 중요한 분기 — 왜 토큰나이저를 두 개 쓰는가

베이스라인(03-06번) → 직접 학습한 **chatbot_bpe.model** (vocab 8,000)
전이학습(04-06번) → 각 모델 고유의 **AutoTokenizer** (KoBART vocab 30,000)

사전학습 모델의 임베딩 행렬은 그 모델 토큰나이저가 만드는 vocab index와 1:1로 대응하도록 학습되어 있다. 토큰나이저를 우리가 새로 학습한 것으로 바꾸면 같은 index가 전혀 다른 서브워드를 가리키게 되어, 사전학습

가중치가 사실상 무작위 값과 다름없어진다. 즉 전이학습의 이점 자체가 사라진다. 이 분기가 03번과 04번의 데이터 파이프라인이 서로 다른 근본 이유이며, 07번에서 두 계열을 비교할 때 "토큰라이저가 달라서 불공정한 것 아닌가"라는 질문에 대한 답이기도 하다 — 각 모델에 최적인 토큰라이저를 쓴 상태로 비교하는 것이 공정한 비교다.

3.4 Word2Vec 임베딩 — 단계별 흐름과 결과

토큰라이저 선택과 별개로, "임베딩이 학습되면서 의미 공간이 어떻게 형성되는가"를 단계별로 직접 확인했다. gensim Word2Vec을 사용했고, 입력은 BPE 서브워드가 아니라 형태소 단위로 했다 — 서브워드는 사람이 읽었을 때 의미 단위가 아닌 경우가 많아, 유사 단어 확인처럼 해석 가능성이 중요한 목적에는 부적합하기 때문이다.

단계	내용	결과
1. 형태소 분석	train의 Q, A 전체를 Okt로 토큰 리스트로 변환	18,800 문장
2. vocab 구축	gensim이 내부적으로 단어 사전 생성	6,048 단어
3. 학습 쌍 생성	각 중심 단어와 window 내 주변 단어 쌍 생성	(내부 처리)
4. 학습	skip-gram + negative sampling, 100차원	word2vec.model 저장
5. 검증	코사인 유사도로 유사 단어 확인	아래 표

표 3-3. Word2Vec 임베딩 학습의 단계별 흐름.

하이퍼파라미터 선택 근거

- skip-gram(sg=1): CBOW는 자주 나오는 단어 학습에 유리하지만, 챗봇 대화는 구어체 표현이 다양하고 저빈도 단어가 많다. skip-gram은 중심 단어 하나로 여러 주변 단어를 예측하므로 저빈도 단어도 학습 기회를 더 얻는다.
- vector_size=100: 코퍼스가 약 1.9만 문장으로 크지 않아, 300차원 같은 큰 임베딩은 과적합·희소성 위험이 있다.
- workers=1: gensim 기본값(workers=3)은 멀티스레드로 학습해 **seed**를 고정해도 실행할 때마다 벡터가 미세하게 달라진다. 이 임베딩으로 초기화하는 06번 실험(A1)의 결과를 재현 가능하게 하려면 단일 스레드 고정 필요하다 — 실제로 두 번 학습해 벡터가 완전히 일치함을 확인했다.

기준 단어	코사인 유사도 상위 5개
사랑	인정(0.778), 존중(0.733), 노력(0.729), 그대로(0.719), 최선(0.718)
이별	폭풍(0.832), 환승(0.802), 만남(0.798), 재회(0.797), 4년(0.780)
고민	혼전순결(0.783), 임(0.768), 느낌(0.762), 심해(0.761), 맞을거예요(0.752)
친구	오빠(0.832), 가족(0.827), 여자애(0.814), 친(0.806), 남사(0.802)

표 3-4. 학습된 임베딩의 유사 단어. "이별-재회-환승-만남", "친구-오빠-가족"처럼 의미적으로 같은 맥락의 단어가 모인 것을 확인할 수 있다.

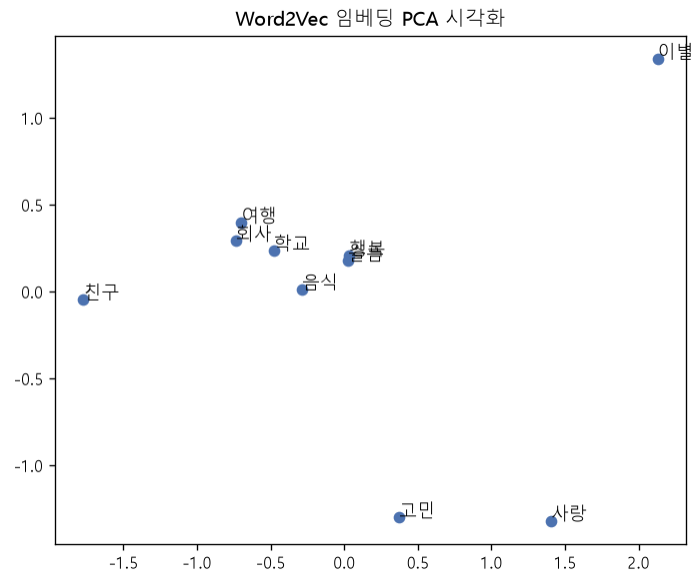


그림 3-1. 주요 단어 임베딩의 PCA 2차원 투영.

이 임베딩은 전이학습 모델(자체 임베딩 보유)에는 쓰이지 않지만, 06번에서 베이스라인 임베딩 초기화 실험(랜덤 vs Word2Vec)의 재료로 이월된다.

4. [03] 베이스라인 모델 — 구조 선택의 근거

4.1 인코더-디코더 vs 디코더 전용

Q→A는 조건부 생성 문제다. 즉 기계번역·요약과 같은 구조적 성격을 가진다. 두 가지 선택지를 놓고 비교했다.

구조	작동 방식	이 과제에서의 평가
인코더-디코더	인코더가 Q 전체를 읽어 압축 표현을 만들고, 디코더가 cross-attention으로 그 표현에 조건화하며 A를 한 토큰씩 생성	채택. Q와 A의 역할 구분이 구조에 직접 반영됨
디코더 전용 (GPT류)	Q와 A를 한 시퀀스로 이어붙여 다음 토큰 예측만으로 학습	소스/타겟 경계를 데이터만으로 학습해야 해 1만 건 규모에서는 상대적으로 불리하다고 판단

표 4-1. 모델 구조 선택. Q와 A는 역할이 뚜렷이 다른 두 시퀀스라는 점이 결정 근거다.

4.2 왜 원 논문보다 작게 만들었는가

원 Transformer 논문과 수업 예제는 d_model 512, layer 6을 쓰지만 이는 WMT 수준(수백만 문장) 데이터를 가정한 크기다. 우리 데이터는 9,400쌍으로 훨씬 작아, 그 크기를 그대로 쓰면 파라미터 대비 데이터가 부족해 과적합과 비효율 학습 위험이 크다. 그래서 d_model과 layer 수를 각각 절반으로 줄였다.

하이퍼파라미터	값	선택 근거
d_model	256	원 논문(512)의 절반 — 데이터 규모에 맞춰 축소
num_layers	인코더 3 / 디코더 3	원 논문(6/6)의 절반
nhead	8	head당 32차원 — 원 논문 비율(512/8=64)의 절반을 유지
dim_feedforward	512	d_model의 2배 (원 논문은 4배, 모델 축소에 비례해 축소)
dropout	0.1	원 Transformer 논문 기본값
MAX_LEN	16	BPE 토큰 길이 95p(10) + BOS/EOS(2) → 8의 배수로 올림
batch size	64	짧은 시퀀스·작은 모델이라 8GB GPU에 여유. 너무 작으면 gradient가 노이즈에 민감, 너무 크면 업데이트 횟수 부족
optimizer	Adam, lr=1e-4	Transformer는 학습 초반 loss가 불안정해 큰 lr보다 안정적인 값 선택
loss	CrossEntropy + label_smoothing 0.1	과확신 억제 — 유사 표현이 많은 챗봇 데이터에서 다양성 저하를 막는 표준 기법
early stopping	patience 5 (최대 30 epoch)	소규모 데이터에서 과적합 억제

표 4-2. 베이스라인 하이퍼파라미터와 선택 근거. 최종 학습 가능 파라미터 수는 6,010,688개다.

임베딩·출력층 가중치 공유(weight tying)

Q와 A가 같은 언어이고 같은 BPE vocab을 공유하므로, 인코더/디코더 임베딩 테이블을 하나로 묶고 출력 projection도 그 임베딩과 가중치를 공유했다. 수업에서 다룬 번역 문제(독일어→영어, 서로 다른 vocab)에서는 쓸 수 없었던 최적화지만, 한국어→한국어인 이 과제에서는 자연스럽게 적용 가능하며 파라미터를 크게 줄여준다.

4.3 1차 학습 결과와 발견된 문제

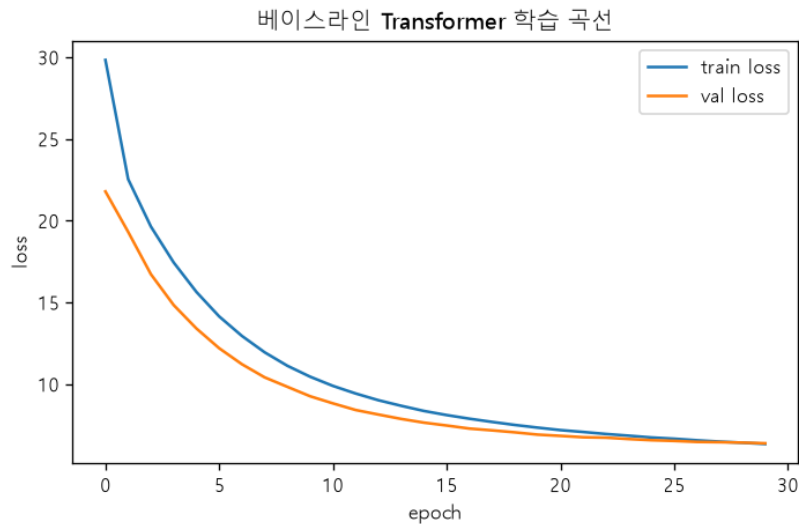


그림 4-1. 베이스라인 1차 학습 곡선 (30 epoch).

30 epoch 예산을 모두 소진하며 train/val loss가 각각 29.82 → 6.3332 / 21.78 → 6.3724로 계속 감소하는 중에 멈췄다. 조기 종료가 걸리지 않았다는 것은 아직 수렴하지 않았다는 신호다. 실제로 greedy decoding 결과에서 같은 토큰을 반복하는 생성 붕괴가 뚜렷하게 관찰됐다.

Q	정답	생성
정장 한 벌 필요할 듯	사면 요긴하긴 할 거 같아요.	마음을 마음을 마음을 마음을 ... (15회 반복)
배터리 겨우 15%야	미리 충전하세요.	많이 많이 많이 많이 ... 생각
청첩장을 찍는 날이 오다니	결혼 얼마 안 남았나봐요.	같이 같이 같이 같이 ... (15회 반복)

표 4-3. 03번 베이스라인의 생성 붕괴 사례.

여기서 바로 모델을 키우거나 다른 기법을 추가하지 않고, 먼저 원인 가설을 세 가지로 정리했다.

- (a) 학습 부족 — epoch 상한 때문에 수렴 전에 멈춘 것
- (b) greedy decoding의 성질 — 매 스텝 1등만 고르는 근시안적 선택이 반복 루프를 만드는 것
- (c) 데이터 규모의 근본적 한계 — 9,400쌍으로는 이 이상이 불가능한 것

이 세 가설을 한꺼번에 건드리면 무엇이 효과가 있었는지 알 수 없다. 그래서 04번에서 (a)부터 다른 조건을 전부 고정한 통제 실험으로 검증하기로 했다. 이것이 04번 노트북 앞부분에 "베이스라인 재실험"이 추가된 이유다.

5. [04] 전이학습 파인튜닝

5.1 통제 실험 — epoch 상한만 30 → 150으로

03번과 모델 구조·손실함수·옵티마이저·배치 크기·시드까지 전부 동일하게 유지하고 epoch 상한과 patience만 늘려 재학습했다. 변수를 하나만 바꿨으므로, 결과 변화는 전적으로 학습량 때문이라고 말할 수 있다.

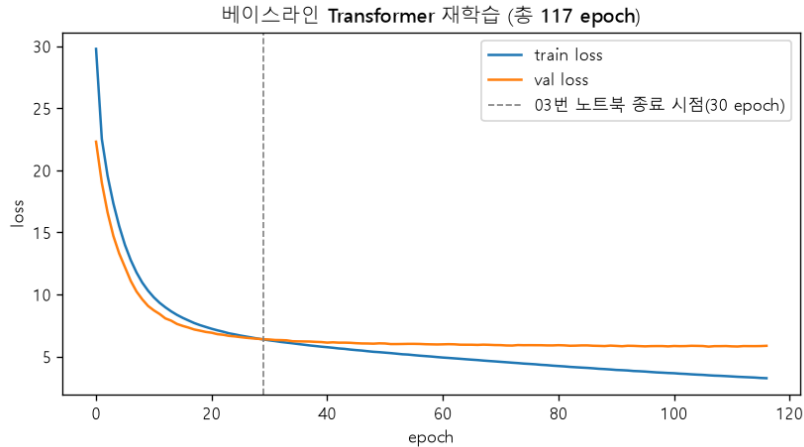


그림 5-1. 베이스라인 재학습 곡선 (117 epoch에서 조기 종료).

구분	epoch	train loss	val loss	생성 결과
03번 (1차)	30 (예산 소진)	6.3332	6.3724	"마음을 마음을 마음을..." 반복 붕괴
04번 (재학습)	117 (조기 종료)	3.30	5.7957	반복이 크게 줄고 정상 문장 등장

표 5-1. 통제 실험 결과. 학습량만 늘렸는데 반복 붕괴가 크게 줄었다.

결론: 가설 (a) 학습 부족이 주된 원인이었다. "배터리 겨우 15%야" → "미리 충전하세요."처럼 정답과 정확히 일치하는 사례도 나왔다. 다만 학습 곡선을 보면 약 30 epoch 지점부터 train과 val loss가 벌어지기 시작해(과적합 시작) val loss는 5.8대에서 평평해진다. 이는 이 데이터·모델 크기 조합의 사실상 성능 한계에 가깝다는 뜻이며, 가설 (c)가 부분적으로 유효함을 시사한다.

5.2 KoBART 파인튜닝 — 전이학습의 효과

전이학습 대상으로 KoBART([gogamza/kobart-base-v2](#))를 선택한 이유는 세 가지다. (1) 한국어로 사전학습된 인코더-디코더 모델이라 우리가 4.1절에서 선택한 구조와 정확히 일치한다. (2) 한국어 챗봇 응답 생성에 공개적으로 검증된 선례가 많다. (3) Hugging Face에서 바로 사용 가능해 재현성이 높다.

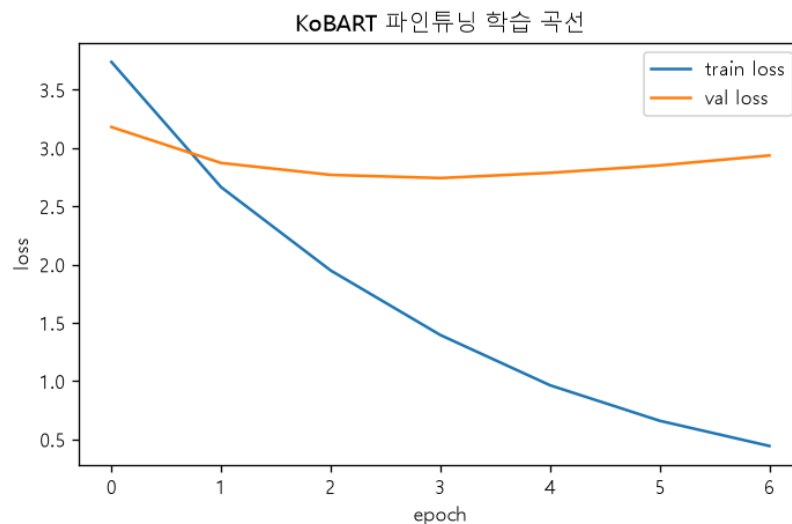


그림 5-2. KoBART 파인튜닝 곡선 (7 epoch에서 조기 종료).

모델	파라미터 수	학습 epoch	best val loss
베이스라인 (직접 구현)	6,010,688	117	5.7957
KoBART 파인튜닝	123,859,968 (약 21배)	7	2.7424

표 5-2. 베이스라인 대비 KoBART. 훨씬 적은 epoch으로 훨씬 낮은 loss에 도달했다.

전이학습의 핵심 가설이 수치로 확인됐다: 사전학습된 언어 지식 덕분에, 같은 데이터에서 베이스라인이 117 epoch로 도달한 수준(5.80)을 KoBART는 단 7 epoch만에 크게 앞질렀다(2.74).

다만 한계도 함께 관찰됐다. train loss가 0.44까지 내려가는 동안 val loss는 5 epoch째부터 반등해 소규모 데이터에 빠르게 과적합하는 경향을 보였다. 이 문제는 7장(하이퍼파라미터 비교)에서 다룬다.

5.3 (중요) 이 노트북에서 발견·수정한 결함

여기에 적은 KoBART 수치는 결함을 수정한 뒤 재실행한 결과다. 최초 실행에서는 이 노트북의 collate 함수가 파인튜닝 타깃에 EOS(문장 끝) 토큰을 붙이지 않아, 모델이 문장을 끝내는 법 자체를 학습하지 못하는 문제가 있었다. 그 결함은 8장(정량 평가)에서 이상 신호로 드러났고 9장(오류 분석)에서 원인이 규명됐다 — 발견 경위와 근거는 9.4절에 상세히 기록했다.

항목	EOS 수정 전	EOS 수정 후
KoBART best val_loss	2.8073	2.7424
생성 예시 ("청첩장을 찍는 날이 오다니")	축하해주요! 예쁘게 찍어주세요. 예쁘게 입으세요. (문장 중간에서 잘림)	축하드려요. (완결)
베이스라인 재학습 (대조군)	117 epoch / 5.7957	117 epoch / 5.7957 (동일 재현)

표 5-3. EOS 수정 전후. 베이스라인은 이 수정과 무관한 별도 파이프라인이므로 수치가 완전히 동일하게 재현됐다 — 다른 변수가 개입하지 않았음을 확인해주는 대조군이다.

6. [05] 디코딩 전략 비교

6.1 왜 디코딩을 별도 축으로 분리했는가

디코딩 전략은 "어떤 모델을 쓰는가"와 독립적인 축이다. 같은 모델이라도 greedy는 안전하지만 뻔한 답을, sampling은 다양하지만 불안정한 답을 낸다. 두 축을 섞으면 "성능 차이가 모델 때문인지 디코딩 때문인지" 설명할 수 없다. 04번에서 이미 KoBART의 greedy 결과에 반복이 관찰됐으므로, 05번은 "이 반복이 디코딩을 바꾸면 실제로 줄어드는가"라는 구체적 질문에서 출발한다.

전략	방식	장점	단점
Greedy	매 스텝 확률 1등 토큰만 선택	빠르고 결정적	근시안적 선택이 누적되어 반복 루프에 빠지기 쉬움
Beam Search	상위 k개 후보 시퀀스를 동시에 유지	전체적으로 더 그럴듯한 문장	확률 높은 쪽으로 수렴해 다양성 낮음, 계산량 증가
Top-k/Top-p Sampling	확률분포에서 무작위 추출하되 저확률 토큰 차단	다양성 확보, 반복 루프 회피	문법·문맥이 어긋날 수 있고 비결정적

표 6-1. 세 디코딩 전략의 이론적 트레이드오프.

하이퍼파라미터 근거

- beam_width=5: NMT 계열에서 4~5가 품질/속도의 무난한 절충점으로 보고되는 값.
- length_penalty=0.7: beam search는 짧은 문장일수록 누적 log-prob이 덜 깎여 유리해지는 편향이 있어, GNMT(Wu et al., 2016) 방식으로 길이의 거듭제곱으로 나눠 보정.
- top_k=50, top_p=0.9, temperature=0.8: nucleus sampling 논문(Holtzman et al., 2019)과 GPT-2 데모에서 널리 쓰인 조합. temperature를 1.0보다 낮춰 분포를 뾰족하게 만들어 산만한 문장을 억제.

6.2 실측 결과 (8개 공통 질문 기준)

전략	distinct-1	distinct-2	연속 반복률
baseline-greedy	0.892	0.893	0.125
baseline-beam	1.000	1.000	0.000
baseline-sampling	1.000	1.000	0.000
kobart-greedy	1.000	1.000	0.000
kobart-beam	1.000	1.000	0.000
kobart-beam + no_repeat_ngram=3	1.000	1.000	0.000

표 6-2. 응답 내부의 반복/다양성 보조 지표 (05번 노트북, 8개 샘플). 전체 test set 기준 정량 비교는 8장에서 다룬다.

베이스라인에서는 디코딩 전략이 실제로 반복을 줄인다. greedy에는 여전히 반복이 남아 있지만(distinct-2 0.893, 연속 반복률 0.125), beam과 sampling에서는 사라졌다. 04번의 에폭 확장이 03번의 극단적 반복 붕괴를 이미 크게 해소했고, 남은 부분을 디코딩이 마저 정리하는 구조다.

반면 KoBART는 세 전략 모두 반복이 0이다. 이것은 중요한 재해석을 요구한다.

반복은 디코딩 문제가 아니라 학습 파이프라인의 결함이었다

이 실험의 최초 실행에서는 KoBART에 구(phrase) 단위 반복이 뚜렷했고(kobart-beam distinct-2 0.940, 연속 반복률 0.125), **no_repeat_ngram_size=3**을 넣어야 해소됐다. 그래서 당시에는 "beam search 자체보다 반복 억제 장치가 더 결정적"이라고 결론지었다.

그러나 9.4절에서 그 반복의 근본 원인이 타깃 EOS 누락임을 규명했다. 모델이 문장을 끝내지 못해

max_length까지 계속 생성하다 보니 같은 구를 반복했던 것이다. 결함을 수정하고 재학습하자 디코딩 전략과 무관하게 반복이 완전히 사라졌고, no_repeat_ngram은 억제할 반복이 없어 사실상 무효(no-op)가 됐다.

즉 최초 결론의 no_repeat_ngram은 학습 결함을 가리는 임시방편이었다. 디코딩 설정으로 증상을 덮는 대신 원인을 찾아 고친 것이 올바른 해법이었다.

Q	정답	KoBART (beam5)
이젠 진짜 진짜 안녕	맘 고생 많았어요.	맘고생 많았어요. (거의 일치)
배터리 겨우 15%야	미리 충전하세요.	충전하고 만나요.
오늘 생각보다 춥네	감기 조심하세요.	마음 감기조심하세요. (sampling: 감기 조심하세요.)
청첩장을 찍는 날이 오다니	결혼 얼마 안 남았나봐요.	예쁘게 찍고 오세요.

표 6-3. EOS 수정 후 KoBART 생성 예시. 수정 전에는 같은 질문에 "축하해주요! 예쁘게 찍어주세요. 예쁘게 입으세요."처럼 여러 문장이 이어지다 잘렸다.

표본이 8건이므로 전체 1,175건 기준 검증은 8.3절에서 수행한다. 모델 비교표에서는 디코딩을 베이스라인 = beam5, 전이학습 = beam5 + no_repeat_ngram=3으로 고정했다 — 후자는 현재 무효에 가깝지만 KE-T5 등 다른 모델과 조건을 동일하게 맞추기 위해 유지했다.

7. [06] 하이퍼파라미터 및 모델 비교 실험

앞선 노트북들에서 "나중에 확인하자"로 미뤄둔 질문 네 가지를, 각각 다른 조건을 고정하고 하나만 바꾸는 통제 실험으로 확인했다.

7.1 Part A — 베이스라인 임베딩 초기화와 learning rate

Word2Vec 초기화를 적용하는 과정에서의 문제와 해결

02번에서 학습한 Word2Vec은 형태소 단위 vocab(6,048개)이고 베이스라인은 BPE 서브워드 vocab(8,000개)이다. 토큰화 단위 자체가 달라 index가 대응되지 않는다. 그래서 다음과 같이 처리했다.

- BPE piece에서 SentencePiece의 단어 시작 표시()를 제거한 문자열이 Word2Vec vocab에 그대로 존재할 때만 벡터를 가져옴 → 8,000개 중 4,272개 매칭(커버리지 53.4%)
- Word2Vec은 100차원, 베이스라인은 256차원이라 고정 랜덤 투사 행렬(100→256)로 차원을 맞춤
- 매칭되지 않는 나머지 토큰은 기존 랜덤 초기화 유지

config	변경한 것	best val loss	epoch
A0 (통제군)	random init, lr=1e-4	5.9692	60 (예산 소진)
A1	Word2Vec init, lr=1e-4	5.8180	60 (예산 소진)
A2	random init, lr=3e-4	5.7364	42 (조기 종료)

표 7-1. 베이스라인 하이퍼파라미터 실험 (공통 예산: 최대 60 epoch, patience 6).

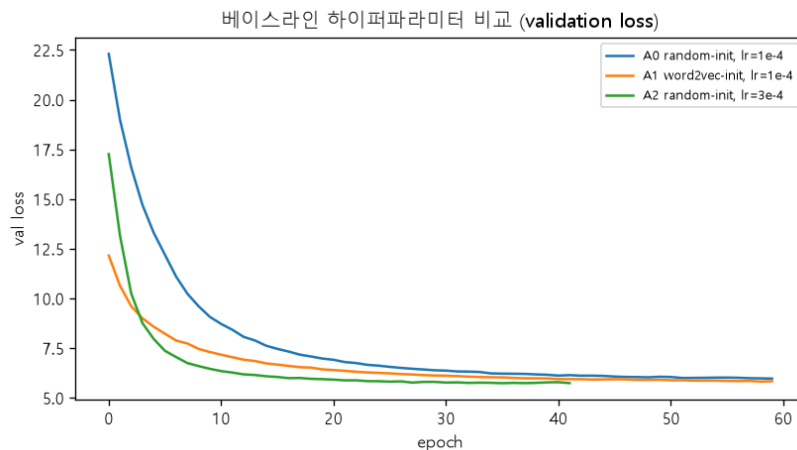


그림 7-1. 베이스라인 세 configuration의 validation loss.

- 임베딩 초기화: Word2Vec 초기화가 랜덤보다 val_loss가 낮았다(5.8180 vs 5.9692). 53.4%만 매칭됐는데도 도움이 된다는 뜻이다. 다만 생성 예시에서는 A1 쪽에 반복이 더 두드러졌다 — loss가 낮다고 생성 품질이 따라 좋아지는 것은 아니라는 사례이며, 8장에서 정량적으로 재확인된다.
- learning rate: lr=3e-4(A2)가 세 config 중 val_loss가 가장 낮았고 유일하게 조기 종료가 걸렸다(42 epoch). 다만 생성 예시에 불안정한 출력도 섞여 있어, lr을 높이면 수렴은 빨라지지만 학습이 불안정해질 위험이 함께 커진다는 트레이드오프를 재확인했다.

7.2 Part B — KoBART 과적합 억제 시도

04번에서 KoBART가 5 epoch째부터 val loss가 반등하는 것을 관찰했으므로, 이를 완화하는 표준적 방법 두 가지를 시도했다: (B1) lr을 낮춰 사전학습 지식을 덜 흔들기, (B2) 동일 lr에 linear warmup + decay 스케줄 추가하기.

config	설정	best val loss	비고
B0 (04번)	lr=5e-5 고정	2.7424	7 epoch 조기 종료
B1	lr=2e-5 고정	2.7994	7 epoch 예산 소진
B2	lr=5e-5 + warmup/decay	2.7479	7 epoch 예산 소진

표 7-2. KoBART 하이퍼파라미터 실험.

세 config가 2.74~2.80으로 매우 근접했다. 04번 설정(B0)이 여전히 가장 낮지만 B2와의 차이는 0.006에 불과해 실질적 차이로 보기 어렵다. 또한 B1·B2는 7 epoch 예산을 다 쓰고도 조기 종료가 걸리지 않았다 — 아직 수렴 중이었다는 뜻이므로 "같은 예산에서의 비교"라는 조건을 감안해 해석해야 한다. 최종 판단은 8장의 정량 지표로 내린다.

7.3 Part C — KE-T5 비교와 학습 중 발생한 문제

fp16에서 NaN 발산 문제와 해결

KE-T5를 fp16 mixed precision으로 학습하자 loss가 NaN으로 발산했다. T5 계열은 원래 TPU에서 bf16으로 사전학습되어 활성값의 동적 범위가 크고, fp16의 좁은 지수 범위를 넘어서는 것으로 알려진 문제다. bf16 autocast로 전환해 해결했다 — bf16은 fp32와 지수 범위가 같아 오버플로우 위험이 없고 GradScaler 없이도 안정적이다.

모델	파라미터 수	best val loss	생성 경향
KoBART (B2)	123,859,968	2.7479	질문 주제를 따라가는 완결된 단문
KE-T5	247,479,552 (약 2배)	4.6432	거의 모든 질문에 "올어도 괜찮아요." — 모드 붕괴

표 7-3. 동일 레시피(lr=5e-5 + warmup, 7 epoch)에서의 KoBART vs KE-T5.

파라미터가 KoBART의 약 2배인데도 같은 예산에서 훨씬 못 미쳤다. KE-T5의 사전학습 목적함수(span corruption)가 BART 계열보다 생성형 파인튜닝에 더 많은 적응 시간을 요구하기 때문으로 보인다. 중요한 것은 "모델이 크다고 좋은 것이 아니라, 사전학습 방식과 과제의 궁합, 그리고 주어진 학습 예산이 함께 작용한다"는 점이며, 이 모드 붕괴 사례는 9장 오류분석에서 다시 다룬다.

7.4 EOS 수정이 이 실험에 미친 영향 — 통제가 지켜졌다는 증거

이 장의 수치는 모두 9.4절에서 규명한 EOS 결함을 수정한 뒤 재실행한 결과다. 수정 전후를 나란히 놓으면, 결함이 있던 모델만 정확히 골라서 변했다는 것이 드러난다.

config	EOS 수정 전	EOS 수정 후	변화
A0 / A2 (베이스라인)	5.9692 / 5.7364	5.9692 / 5.7364	완전히 동일
B1 (KoBART lr=2e-5)	3.2028	2.7994	0.40
B2 (KoBART lr=5e-5+warmup)	3.1311	2.7479	0.38
KE-T5	4.6432	4.6432	완전히 동일

표 7-4. EOS 수정 전후 비교. 이 표 자체가 원인 규명이 정확했음을 뒷받침하는 증거다.

- 베이스라인(A계열)은 값이 하나도 바뀌지 않았다 — BPE 파이프라인(03번 `encode_text`)을 쓰므로 원래부터 EOS가 있었고, 이번 수정과 무관하다.
- KE-T5도 값이 정확히 같다 — KE-T5 토큰라이저는 타깃 인코딩 시 EOS를 자동으로 붙이기 때문에 이번 수정이 아무 효과가 없는(no-op) 경우였다. 의도하지 않게 얻은 자연 대조군인 셈이다.
- 오직 KoBART 계열만 개선됐다 — 결함이 있던 바로 그 모델만 영향을 받았다.

8. [07] 정량 평가

8.1 평가 설계 — 무엇을 왜 재는가

여기까지의 판단 근거는 validation loss와 8개 샘플 정성 비교뿐이었다. 그런데 loss는 "정답 토큰에 얼마나 높은 확률을 줬는가"를 잘 뿐이고, 게다가 teacher forcing으로 계산되므로 "앞 토큰이 전부 정답일 때"의 성능이다. 자기가 생성한 토큰을 다시 입력으로 쓰는 실제 추론 상황과는 차이가 있다(exposure bias). 그래서 실제 생성 결과를 직접 채점했다.

지표를 4종으로 구성한 이유

챗봇 응답은 "정답이 하나가 아닌" 과제다. "배고파"에 "뭐 좀 드세요"와 "얼른 식사하세요"는 둘 다 좋은 답이지만 표면이 겹치지 않는다. 표면 유사도 하나로는 모델을 오판하기 쉬우므로, 서로 다른 실패를 잡아내는 지표를 함께 봤다.

지표	재는 것	이 지표가 잡아내는 실패
BLEU	n-gram 정밀도 (+짧은 문장 패널티)	정답과 전혀 다른 말을 하는 경우
ROUGE-L	LCS 기반 재현율 중심 F1	정답의 핵심 내용을 빠뜨리는 경우
Distinct-1/2	corpus 전체에서 서로 다른 n-gram 비율	모든 질문에 같은 답만 하는 붕괴
빈 응답 비율	아무것도 생성하지 못한 비율	생성 자체의 실패
(보조) 연속 반복률	같은 토큰이 연달아 나온 응답의 비율	"마음을 마음을 마음을" 류 반복
(보조) 평균 길이	응답의 평균 형태소 수	BLEU가 길이에 왜곡되는 것을 감지

표 8-1. 평가 지표 구성. Distinct-n을 corpus 단위로 계산한 것과 평균 길이를 함께 본 것이, 9장에서 두 가지 중대한 문제를 찾아내는 결정적 역할을 한다.

왜 형태소 단위로 채점하는가

BLEU/ROUGE는 띄어쓰기가 곧 단어 경계인 언어를 가정한 지표다. 한국어를 어절 기준으로 채점하면 조사 하나 차이로 완전 불일치가 된다 ("충전하세요" vs "충전 하세요"가 0점). 반대로 음절 기준은 과도하게 관대하다. 02번에서 "형태소가 의미 보존과 길이의 절충점"이라는 결론을 냈으므로, 평가에서도 같은 원칙을 적용해 정답·생성문을 모두 Okt 형태소로 분해한 뒤 채점했다. 따라서 아래 절대값은 음절 기준보다 낮고 어절 기준보다 높게 나오는 것이 정상이며, 표의 목적은 모델 간 상대 비교다.

8.2 모델 9종 비교 결과

test set 1,175건 전체에 대해, 05번 결론에 따라 디코딩을 고정(베이스라인 beam5 / 전이학습 beam5 + no_repeat_ngram=3)하고 평가했다. test 정답의 평균 길이는 6.46형태소다.

모델	계열	BLEU	ROUGE-L	Dist-1	Dist-2	반복률	평균길이
KoBART B2 lr5e-5+warmup (06)	전이학습	7.90	25.14	0.223	0.510	0.017	6.22
KoBART lr5e-5 (04)	전이학습	7.72	24.90	0.208	0.486	0.024	6.28
KoBART B1 lr2e-5 (06)	전이학습	6.72	24.18	0.218	0.496	0.031	6.21
A2 random lr3e-4 (06)	베이스라인	4.87	22.78	0.110	0.239	0.003	5.18
baseline-114ep (04)	베이스라인	4.87	22.99	0.128	0.303	0.014	5.18
baseline-30ep (03)	베이스라인	2.64	22.43	0.027	0.070	0.083	4.54
A1 word2vec lr1e-4 (06)	베이스라인	2.50	21.92	0.080	0.201	0.017	5.14
A0 random lr1e-4 (06)	베이스라인	2.21	21.04	0.048	0.100	0.008	5.18
KE-T5 lr5e-5+warmup (06)	전이학습	0.09	19.47	0.002	0.003	0.000	4.00

표 8-2. 전체 모델 비교 (test 1,175건, 형태소 단위 채점). 빈 응답 비율은 9개 모델 모두 0.000이었다.

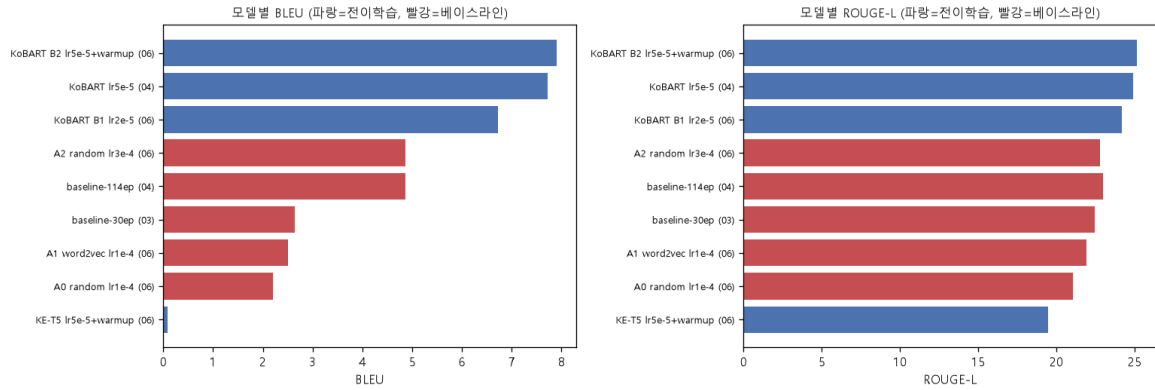


그림 8-1. 모델별 BLEU와 ROUGE-L.

결과 1 — 전이학습이 모든 지표에서 명확히 앞선다

지표	최고 전이학습 (KoBART B2)	최고 베이스라인 (114ep)	차이
BLEU	7.90	4.87	+62%
ROUGE-L	25.14	22.99	+9%
Distinct-2 (다양성)	0.510	0.303	+68%
평균 길이 (정답 6.46)	6.22	5.18	정답에 더 근접

표 8-3. 전이학습 대 베이스라인의 최고 성능 비교.

KoBART 계열 3종이 베이스라인 5종 전체를 앞선다. 특히 주목할 점은 정확도(BLEU/ROUGE)와 다양성(Distinct-n)이 동시에 높다는 것이다. 보통 이 둘은 상충하는데(8.3절의 sampling 사례 참조), 전이학습은 두 축을 함께 끌어올렸다. 사전학습으로 이미 언어 표현을 폭넓게 익힌 모델이 소규모 데이터에 적응할 때 얻는 이점이 그대로 드러난 결과다.

중요: 이 결론은 9.4절에서 규명한 EOS 결함을 수정한 이후의 것이다. 수정 전에는 정반대 결과(베이스라인 4.87 > KoBART 3.83)가 나왔고, 그 모순을 추적한 과정이 이 프로젝트의 가장 중요한 발견이다. 8.5절과 9.4절에 상세히 기록했다.

결과 2 — val_loss 순위와 생성 품질 순위는 여전히 부분적으로만 일치한다

모델	06번 val_loss	07번 BLEU	일치 여부
KoBART B0 (04번)	2.7424 (1위)	7.72 (2위)	불일치
KoBART B2 (warmup)	2.7479 (2위)	7.90 (1위)	불일치
KoBART B1 (lr=2e-5)	2.7994 (3위)	6.72 (3위)	일치
A2 (lr=3e-4)	5.7364 (베이스라인 1위)	4.87 (공동 1위)	일치
A1 (Word2Vec 초기화)	5.8180 (2위)	2.50 (4위)	불일치

표 8-4. val_loss 순위와 BLEU 순위 비교.

val_loss가 가장 낮았던 B0(2.7424)보다 B2(2.7479)의 BLEU가 더 높았다. 차이는 작지만 방향이 반대다. A1(Word2Vec 초기화)은 더 뚜렷하다 — val_loss로는 A0보다 확실히 나았지만(5.8180 vs 5.9692) BLEU는 2.50으로 baseline-30ep(2.64)보다도 낮다. "loss 최저값이 최선의 생성 모델을 보장하지 않는다"는 것이 이 프로젝트에서 반복적으로 확인된 사실이며, 07번을 별도 단계로 둔 이유가 바로 이것이다.

8.3 디코딩 전략 — 8개 샘플 결론의 전체 검증

모델	디코딩	BLEU	ROUGE-L	Distinct-2	연속반복률	평균길이
baseline-114ep	greedy	3.05	20.61	0.425	0.121	6.65

모델	디코딩	BLEU	ROUGE-L	Distinct-2	연속반복률	평균길이
baseline-114ep	beam5	4.87	22.99	0.303	0.014	5.18
baseline-114ep	top-k50/top-p0.9	3.06	20.01	0.563	0.106	6.70
KoBART	greedy	5.91	24.98	0.225	0.026	5.85
KoBART	beam5	7.71	24.68	0.206	0.024	6.33
KoBART	beam5 + no_repeat_ngram3	7.72	24.90	0.208	0.024	6.28
KoBART	top-k50/top-p0.9	5.01	23.47	0.239	0.026	6.04

표 8-5. 디코딩 전략별 전체 test set 성능.

- 05번 결론이 입증됐다. 베이스라인에서 beam5는 greedy 대비 BLEU를 3.05 → 4.87로 60% 끌어올렸고, 연속반복률도 0.121 → 0.014로 거의 없었다. KoBART에서도 beam5가 greedy보다 크게 높다(7.71 vs 5.91).
- no_repeat_ngram의 추가 이득은 0.01에 불과하다(7.71 → 7.72). EOS 수정 후에는 억제할 반복이 거의 없어 이 설정이 사실상 무효가 됐기 때문이다 — 6.2절에서 설명한 "임시방편이었다"는 진단과 일치한다.
- sampling의 트레이드오프가 선명하다. 두 모델 모두 Distinct-2는 가장 높지만 BLEU/ROUGE는 가장 낮다. "다양성과 정답 일치도는 상충한다"는 것이 수치로 확인된다.

8.4 감정 label별 성능

모델	label	n	BLEU	ROUGE-L	Distinct-2
baseline-114ep	0 일상	529	8.05	25.57	0.428
baseline-114ep	1 이별	350	2.46	21.75	0.440
baseline-114ep	2 사랑	296	2.44	19.85	0.423
KoBART	0 일상	529	13.22	28.73	0.607
KoBART	1 이별	350	3.55	23.05	0.505
KoBART	2 사랑	296	3.64	20.22	0.642

표 8-6. 감정 label별 성능 분해.

두 모델 모두 label 0(일상)에서 압도적으로 높고 label 1:2로 갈수록 떨어진다. 01번에서 확인한 데이터 쏠림(일상 5,290 / 이별 3,570 / 사랑 2,963)과 방향이 일치한다. 주목할 점은 KoBART가 모든 label에서 베이스라인을 앞서되 label 0에서 격차가 가장 크다는 것(13.22 vs 8.05)이다. 데이터가 많은 영역에서 사전학습의 이점이 더 크게 발현된다고 해석할 수 있다.

8.5 EOS 수정 전후 — 통제 실험

이 장의 모든 수치는 9.4절에서 규명한 결함을 수정한 뒤의 것이다. 수정 전후를 나란히 놓으면 변화의 크기와 통제가 지켜졌다는 증거를 함께 볼 수 있다.

모델	BLEU 전→후	ROUGE-L 전→후	평균길이 전→후	정상종결% 전→후
KoBART lr5e-5 (04)	3.83 → 7.72	17.54 → 24.90	12.32 → 6.28	32.3 → 98.0
KoBART B1 lr2e-5	3.26 → 6.72	17.38 → 24.18	12.23 → 6.21	35.1 → 98.6
KoBART B2 lr5e-5+warmup	3.38 → 7.90	17.30 → 25.14	12.31 → 6.22	34.6 → 98.4
KE-T5 (자연 대조군)	0.09 → 0.09	19.47 → 19.47	4.00 → 4.00	100.0 → 100.0
baseline-114ep (대조군)	4.87 → 4.87	22.99 → 22.99	5.18 → 5.18	99.6 → 99.6

표 8-7. EOS 수정 전후 비교 (전체 test set).

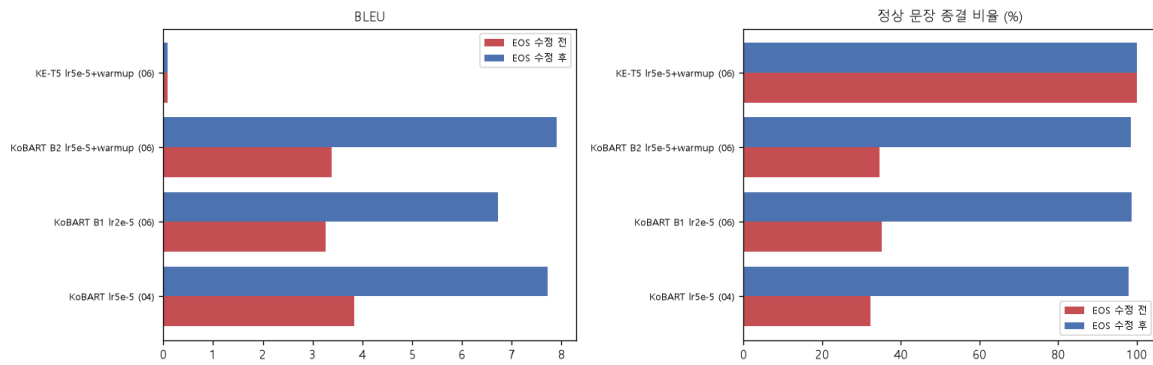


그림 8-2. EOS 수정 전후 BLEU와 정상 문장 종결 비율.

EOS 하나를 고친 것만으로 KoBART의 BLEU가 2배 이상 올랐다(3.83 → 7.72). 평균 생성 길이는 12.32 → 6.28형태소로 정답(6.46)에 거의 정확히 맞춰졌고, 정상 문장 종결률은 32.3% → 98.0%가 됐다.

대조군 두 개가 전혀 변하지 않았다는 점이 이 실험의 신뢰도를 뒷받침한다. 베이스라인은 원래 EOS를 붙이고 있었고, KE-T5는 토크나이저가 EOS를 자동으로 붙여 이번 수정이 무효였다. 즉 결함이 있던 모델만 정확히 골라서 개선됐다.

9. [08] 오류 분석

9.1 분석을 두 층으로 나눈 이유

오류에는 규칙으로 기계적으로 셀 수 있는 것과, 사람이 읽어야만 판정되는 것이 섞여 있다. 이 둘을 섞으면 "대충 이런 것 같다"는 인상비평이 되기 쉽다. 그래서 명확히 분리했다.

- A층 (자동 탐지): 빈 응답, 단일 토큰 반복, 구 단위 반복, 질문 복사 — 정답을 몰라도 생성문만 보고 객관적으로 판정 가능한 명백한 결함.
- B층 (수동 판정): 문장은 멀쩡한데 정답과 표면이 전혀 겹치지 않는 경우. 진짜 오답일 수도, 정답과 다를 뿐 타당한 답일 수도 있어 자동 판정이 원리적으로 불가능하다. 표본을 직접 읽어 분류했다.

A층의 각 규칙은 임의로 만든 것이 아니라, 03~05번에서 실제로 눈으로 본 실패를 규칙으로 옮긴 것이다. 예를 들어 "단일 토큰 반복"은 03번의 "마음을 마음을 마음을" 붕괴에서 왔다.

9.2 A층 — 모델별 오류 유형 분포 (단위 %)

모델	단일 토큰 반복	구 단위 반복	질문 복사	표면 불일치	결함 없음
baseline-30ep (03)	8.3	0.5	0.0	3.7	87.6
baseline-114ep (04)	1.4	0.2	0.0	4.1	94.3
A0 random lr1e-4 (06)	0.8	0.3	0.2	4.7	94.1
A1 word2vec lr1e-4 (06)	1.7	1.2	0.3	4.0	92.9
A2 random lr3e-4 (06)	0.3	0.1	0.0	4.1	95.5
KoBART lr5e-5 (04)	2.4	0.9	0.9	6.2	89.7
KoBART B1 lr2e-5 (06)	3.1	0.7	0.9	5.3	90.0
KoBART B2 lr5e-5+warmup (06)	1.7	0.9	1.2	6.1	90.1
KE-T5 lr5e-5+warmup (06)	0.0	0.0	0.0	4.3	95.7

표 9-1. 전체 test set 1,175건 기준 오류 유형 분포. 빈 응답은 모든 모델에서 0건.

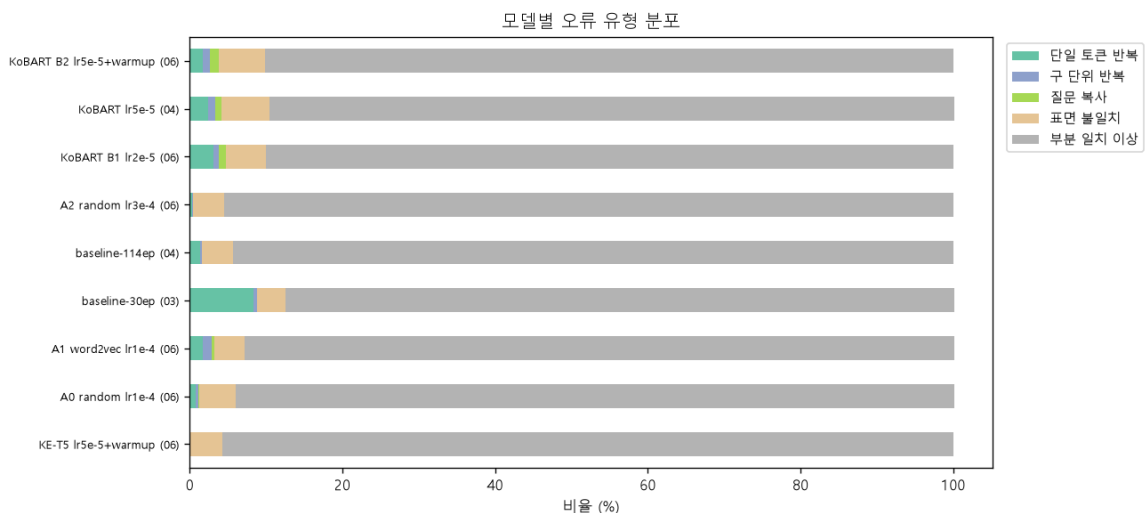


그림 9-1. 모델별 오류 유형 분포.

전이학습 계열의 "표면 불일치"가 베이스라인보다 높게(5.3~6.2% vs 3.7~4.7%) 보이는데, 9.5절의 수동 판정 결과를 보면 이는 오류가 아니라 지표의 한계에 가깝다.

9.3 개선 추적 — 무엇을 고쳤더니 무엇이 줄었는가

이것이 이 프로젝트에서 요구되는 핵심 증명이다. 시도한 변경과 그 결과를 같은 기준으로 정리했다.

변경	단일 토큰 반복	구 단위 반복	ROUGE-L	Distinct-1	판정
03 → 04 (epoch 30 → 117)	8.3% → 1.4%	3.1% → 0.9%	22.4 → 23.0	0.027 → 0.128	성공
04 → 06 A1 (Word2Vec 초기화)	1.4% → 1.7%	0.9% → 1.8%	23.0 → 21.9	0.128 → 0.080	악화
04 → 06 A2 (lr 1e-4 → 3e-4)	1.4% → 0.3%	0.9% → 0.1%	23.0 → 22.8	0.128 → 0.110	혼재
베이스라인 → KoBART	1.4% → 2.4%	0.9% → 0.9%	23.0 → 24.9	0.128 → 0.208	성공
KoBART EOS 결함 수정	—	22.0% → 0.9%	17.5 → 24.9	0.141 → 0.208	성공 (최대)

표 9-2. 변경별 오류 유형 변화 추적.

가장 큰 개선 두 가지는 모두 "모델을 바꿔서"가 아니라 "잘못된 것을 고쳐서" 얻었다.

- 03 → 04 에폭 확장: 단일 토큰 반복 8.3% → 1.4%(6분의 1), Distinct-1 0.027 → 0.128(4.7배)
- EOS 결함 수정: 구 단위 반복 22.0% → 0.9%, ROUGE-L 17.5 → 24.9

특히 baseline-30ep의 Distinct-1 0.027은 1,175개 질문에 대한 응답의 어휘가 거의 겹친다는 뜻으로, 사실상 모든 질문에 비슷한 답을 하는 상태였다. 이 실패는 A층 규칙(반복·빈 응답)만으로는 잡히지 않고 corpus 단위 Distinct-n으로만 드러난다 — 8.1절에서 지표를 4종으로 구성한 설계가 여기서 값을 했다.

9.4 핵심 발견 — 지표와 인상이 어긋났을 때 무엇을 의심할 것인가

이 절은 이 프로젝트에서 가장 중요한 발견을 다룬다.

발견 경위

07번 최초 실행 결과는 사전 예상과 정반대였다. 사전학습 KoBART(BLEU 3.83 / ROUGE-L 17.54)가 처음부터 학습한 베이스라인(4.87 / 22.99)에 졌다. 당시 04번 실행에서 KoBART의 val_loss(2.81)가 베이스라인(5.80)보다 훨씬 낮았던 것과도 어긋났고, 눈으로 본 생성 문장은 분명 KoBART 쪽이 유창했다.

정성적 인상과 정량 지표가 어긋날 때 가능한 설명은 둘 중 하나다.

- (a) 지표가 잘못 재고 있다 — 챗봇은 정답이 여러 개라 표면 유사도가 품질을 대변하지 못한다
- (b) 파이프라인에 결함이 있다 — 모델이 실제로 무언가를 잘못하고 있고, 지표가 그것을 정확히 잡아냈다

"챗봇은 정답이 여러 개니까 지표의 한계겠지"라며 (a)로 결론내고 넘어가는 것이 쉬운 길이었다. 그러나 그렇게 하면 진짜 문제를 놓친다. (b)부터 검증했고, 실제로 결함이 있었다.

단서 1 — 길이와 문장 종결

모델	생성 평균 형태소	정답 평균 형태소	정상 문장 종결 비율
baseline-114ep (04)	5.18	6.46	99.6%
KoBART lr5e-5 (04)	12.32	6.46	32.3%
KE-T5 lr5e-5+warmup (06)	4.00	6.46	100.0%

표 9-3. (수정 전) 생성 길이와 문장 완결성 진단.

KoBART는 정답의 약 2배 길이로 생성하면서 정상적으로 문장이 끝나는 비율이 32.3%에 불과했다. 베이스라인은 99.6%가 제대로 끝난다. 이건 "지표의 한계"가 아니라 모델이 문장을 끝내지 못하는 실제 결함이다.

단서 2 — 왜 끝내지 못하는가

생성을 멈추는 신호는 EOS(문장 끝) 토큰이다. 04/06번 파인튜닝에 실제로 들어간 타깃 토큰 열을 직접 재현해 확인했다.

```
tokenizer(text_target=["미리 충전하세요."])
```

→ 토큰: [' 미리', ' 충전', '하', '세요.']

→ 마지막 토큰이 EOS인가? 아니오 (eos_token_id = 1이 열에 존재하지 않음)

gogamza/kobart-base-v2 토크나이저는 타깃을 인코딩할 때 EOS를 자동으로 붙이지 않는다. 04/06번 파인튜닝은 이를 보정하지 않았으므로, 모델은 학습 과정에서 "문장을 끝내는 법"을 배울 기회 자체가 없었다. 그래서 생성 시 EOS를 내지 않고 max_length까지 계속 생성하다 강제로 잘린다.

단서 3 — 대안 가설의 배제

"타깃이 max_length에서 잘리면서 EOS가 함께 사라진 것 아닌가"라는 다른 설명도 가능하다. 직접 확인했다.

- KoBART 토큰 기준 타깃 길이: 평균 6.81, 95 percentile 12, 최대 30
- max_length=16에서 잘리는 타깃: 77건 / 9,400건 (0.8%)
- → 1% 미만이므로 이 가설로는 68%의 미완결 응답을 설명할 수 없다. 대안 가설 기각.

근본 원인: 베이스라인은 03번 **encode_text()**에서 **[BOS] + ids + [EOS]**로 EOS를 명시적으로 붙였다. 같은 처리를 전이학습 쪽에만 빠뜨린 것이 결함의 전부였다.

수정과 재실행

04/06번의 collate 함수에서 타깃 끝에 EOS를 추가하도록 고치고(**encode_targets_with_eos**), 다른 조건은 전부 그대로 둔 채 04 → 05 → 06 → 07 → 08을 재실행했다. 토크나이저에 따라 EOS를 이미 붙이는 경우가 있으므로(KE-T5), 없을 때만 추가하도록 조건을 걸었다. 결과는 8.5절의 표와 같다 — KoBART의 BLEU가 2배 이상 올랐고, 두 대조군은 값이 전혀 변하지 않았다.

이 발견의 의미: 최초 실행의 "베이스라인이 전이학습을 이겼다"는 결과는 전이학습이 이 과제에 부적합하다는 증거가 아니라 파인튜닝 파이프라인의 결함이 만든 결과였다. 지표가 정성적 인상과 어긋났을 때 지표를 의심하는 대신 파이프라인을 의심한 것이 결함을 찾아낸 결정적 판단이었다.

9.5 B층 — 표면 불일치 20건 수동 판정

표면 불일치(정답과 공통 형태소 0개)로 분류된 사례를 대표 모델 2종에서 각 10건씩 직접 읽고 분류했다.

수동 분류	baseline-114ep	KoBART lr5e-5
타당한 대안 응답 (사실상 오류 아님)	2건	8건
문맥 무시 (질문 주제와 무관)	6건	0건
감정 오독	1건	0건
비문/불완전	1건	2건

표 9-4. 표면 불일치 사례의 수동 분류 결과 (각 10건).

두 모델의 실패 성격이 근본적으로 다르다.

모델	실패의 성격	실제 사례
baseline-114ep	진짜 오류 — 문장은 완결되지만 질문과 무관	Q "나 사랑하니?" → "잠시 휴식을 취해보세요." Q "역류성 식도염이 또 도졌어" → "사실적이죠." Q "상 받을 거 같아" → "나쁜 사람이네요." (감정 오독)
KoBART	대부분 지표의 한계 — 정답과 다를 뿐 타당한 답	Q "첫 눈에 반하는게 가능해?" (정답 "당연히 가능하죠.") → "가능합니다!" Q "더 사랑해달래" (정답 "더 많이 표현해주세요.") → "사랑해요!" Q "부업할까?" (정답 "좋아요!") → "부업하시나봐요."

표 9-5. 두 모델의 실패 성격 대비.

중요한 함의: KoBART의 표면 불일치 사례 10건 중 8건이 타당한 응답이었다. 특히 정답 "당연히 가능하죠."에 대한 생성 "가능합니다!"는 의미가 사실상 동일한데 형태소가 하나도 겹치지 않아 ROUGE-L이 0으로 계산된다.

즉 KoBART의 BLEU 7.90은 과소평가된 값이다. 정답이 하나뿐인 데이터로 다대다 대응 과제를 채점하는 방식 자체가 전이학습 모델의 성능을 깎아내리고 있다. 사람 평가나 의미 기반 지표(BERTScore 등)가 필요한 이유다.

9.6 KE-T5 — 규칙 기반 탐지가 놓치는 실패

KE-T5는 A층 기준으로는 가장 깨끗해 보인다 — 명백한 결함 없음 95.7%, 반복 0%, 정상 종결 100%. 그러나 실제로는 거의 모든 질문에 "울어도 괜찮아요." 한 문장만 내놓는 완전한 모드 붕괴 상태다. 이 실패는 07번의 Distinct-1 0.002, BLEU 0.09로만 드러난다.

방법론적 교훈: 생성문 하나만 보고 판정하는 규칙(반복·빈 응답·질문 복사)으로는 "모든 응답이 서로 같은" 실패를 원리적으로 탐지할 수 없다. corpus 단위 다양성 지표를 반드시 함께 봐야 하는 이유다.

9.7 감정 label별 오류

모델	label	n	반복률(%)	표면 불일치(%)	평균 ROUGE-L
baseline-114ep	0 일상	529	0.6	7.0	25.57
baseline-114ep	1 이별	350	0.9	1.4	21.75
baseline-114ep	2 사랑	296	1.4	2.0	19.85
KoBART	0 일상	529	0.2	9.5	28.73
KoBART	1 이별	350	0.9	2.9	23.05
KoBART	2 사랑	296	2.0	4.4	20.22

표 9-6. label별 오류 분해.

ROUGE-L은 label 0에서 가장 높지만 표면 불일치도 label 0에서 가장 높다(7.0~9.5% vs 1.4~4.4%). 일상 대화는 주제가 넓어("더워더워", "역류성 식도염") 정답 표현의 폭이 큰 반면, 이별·사랑 질문은 응답 어휘가 정형적("힘내세요", "축하합니다")이어서 우연히라도 겹치기 쉽기 때문으로 보인다.

10. 결론 및 향후 계획

10.1 이 프로젝트에서 확인한 것

- 전이학습은 실제로 더 낫다 — 모든 지표에서. KoBART B2가 최고 베이스라인 대비 BLEU +62%(7.90 vs 4.87), Distinct-2 +68%(0.510 vs 0.303). 정확도와 다양성을 동시에 끌어올렸다. 학습 비용도 7 epoch vs 117 epoch으로 훨씬 적다.
- 통제 실험은 작동했다. 03 → 04에서 epoch 상한 하나만 바꿔 "반복 붕괴의 원인은 학습 부족"이라는 가설을 검증했고, 단일 토큰 반복 8.3% → 1.4%, Distinct-1 0.027 → 0.128로 확인됐다.
- 가장 큰 성능 개선은 모델이 아니라 결합 수정에서 나왔다. EOS 하나를 고쳐 KoBART의 BLEU가 3.83 → 7.72로 2배 이상 올랐다. 대조군 두 개(베이스라인, KE-T5)가 전혀 변하지 않아 인과가 명확하다.
- 디코딩 전략은 모델과 독립적인 개선 축이다. 같은 가중치로 beam5를 쓰는 것만으로 베이스라인 BLEU가 3.05 → 4.87로 올랐다. 다만 반복 억제 제약(no_repeat_ngram)은 학습 결합을 덮는 임시방편이었고, 원인을 고치자 무효가 됐다.
- loss와 생성 품질은 같은 방향으로만 움직이지 않는다. val_loss 1위였던 B0보다 2위 B2의 BLEU가 높았고, Word2Vec 초기화(A1)도 loss는 낮았지만 반복은 더 많았다.
- 지표 하나로는 모델을 판정할 수 없다. KE-T5는 오류 규칙 기준으로 가장 깨끗했지만 corpus 단위 Distinct-1(0.002)로만 모드 붕괴가 드러났다.

10.2 현재의 한계 — 정직하게

- BLEU 절대값이 낮고, 그중 상당 부분이 지표의 한계다. 9.5절에서 KoBART의 표면 불일치 10건 중 8건이 타당한 응답으로 판정됐다 — 정답이 하나뿐인 데이터로 다대다 대응 과제를 채점하는 구조적 문제다. 현재 수치는 전이학습 모델의 실력을 과소평가하고 있을 가능성이 높다.
- KE-T5는 공정하게 비교되지 않았다. 7 epoch 예산에서 수렴 자체가 미완이었으므로, "KoBART보다 나쁘다"가 아니라 "이 예산에서는 판정 불가"가 정확한 표현이다.
- label 정보를 모델에 전혀 사용하지 않았다. 감정 label을 조건으로 주는 구조는 시도하지 않았다.
- label 1·2 성능이 label 0의 3분의 1 수준이다(BLEU 3.55/3.64 vs 13.22). 데이터 쓸림 대응이 없었다.
- (Q, A)는 같고 label만 다른 행의 처리는 미해결로 남아 있다(2.2절).
- 하이퍼파라미터 탐색 범위가 좁다. 예산 제약으로 각 축을 2~3개 값만 비교했고, KoBART B1/B2는 7 epoch 예산을 다 쓰기도 수렴 전이었다.

10.3 다음 단계 (우선순위 순)

순위	할 일	근거	기대 효과
1	사람 평가 또는 의미 기반 지표(BERTScore 등) 추가	9.5절	현재 지표가 놓치는 "타당한 대안 응답"을 제대로 반영 — 가장 큰 왜곡을 제거
2	베이스라인의 "문맥 무시" 오류 개선	9.5절 (실패 10건 중 6건)	베이스라인 실패의 주종 해소
3	KoBART 학습 예산 확대 후 B0/B1/B2 재비교	7.2절 (수렴 전 종료)	하이퍼파라미터 결론 확정
4	label 1·2 성능 보강 (가중 샘플링 등)	8.4·9.7절	데이터 쓸림 대응
5	KE-T5 학습 예산 확대 또는 비교군에서 제외	9.6절	모델 비교 축의 결론 확정

표 10-1. 2차 작업 계획.

부록 A. 실행 환경 및 재현 방법

항목	내용
OS	Windows 11
Python	3.11 (프로젝트 전용 가상환경 .venv)
PyTorch	2.11.0+cu128 (CUDA 12.8)
GPU	NVIDIA GeForce RTX 4070 Ti
Java	Eclipse Temurin JDK 17 (KoNLPy Okt의 JVM 의존성)
주요 라이브러리	transformers 5.17.0, tokenizers 0.23.2, sentencepiece 0.2.2, gensim 4.4.0, konlpy 0.6.0, sacrebleu 2.6.0, rouge-score 0.1.2
재현 절차	① .venv 생성 후 requirements.txt 설치 (dataclasses==0.8은 Python 3.11에서 설치 불가·불필요하므로 제외) ② JDK 설치 ③ notebooks/01 → 08 순서대로 실행
주의	노트북은 상대경로(../dataset, ../checkpoints, ../results)를 사용하므로 notebooks/ 디렉토리를 작업 디렉토리로 실행해야 한다

부록 B. 산출물 목록

경로	내용
notebooks/01~08	전 과정 코드와 실행 결과 (각 노트북 말미에 실측 수치 기반 요약 포함)
dataset/processed/	정제·분할 데이터, BPE 모델, Word2Vec 모델, 학습곡선 그래프
checkpoints/	베이스라인 5종(.pt), 전이학습 4종(모델+토큰나이저)
results/metrics_all_models.csv	모델 9종 × 지표 7종 비교표
results/metrics_decoding.csv	디코딩 전략별 전체 test set 성능
results/metrics_by_label.csv	감정 label별 성능 분해
results/eos_fix_comparison.csv	EOS 수정 전후 비교 (통제 실험 결과)
results/predictions_all.csv	9개 모델 × 1,175건 전체 생성 결과 (10,575행)
results/error_analysis.csv	생성 결과별 오류 유형 판정
results/error_improvement_tracking.csv	변경 단계별 오류 변화 추적
results/length_diagnosis.csv	생성 길이·문장 종결 진단 (9.4절 근거)
results/before_eos_fix/	EOS 수정 전 결과 원본 (비교용 보존)
PROGRESS_LOG_2026-09-19.pdf	이번 세션 진행 로그
report/결과보고서.pdf	본 문서

모든 수치는 2026-09-19 실행 결과 기준이며, 노트북 01~08의 출력과 일치한다. EOS 결함 수정 전 결과는 비교 목적으로 results/before_eos_fix/에 보존되어 있다.